



SAI VIDYA INSTITUTE OF TECHNOLOGY

(Affiliated to VTU, Belagavi, Approved by AICTE, New Delhi and Govt. of Karnataka)

Accredited by NBA, New Delhi (CSE, ISE, ECE), NAAC – ‘A’ Grade

Rajanukunte, Bengaluru-560064

Tel: 080-2846 8196, Fax: 2846 8193 / 98, Web: www.saividya.ac.in



MOTTO

“Learn to lead”

VISION

Contribute dedicated, skilled, intelligent engineers and business administrators to architect strong India and the world.

MISSION

M1: To provide quality education and skilled training to produce dedicated engineers and managers.

M2: To promote research, innovation, and ethical practices by creating supportive environment.

M3: To undertake collaborative projects with academia and industry that transform young minds into Socially Responsible Citizen and Globally Competent Professionals.

M4: To enhance personality traits which lead to entrepreneurship qualities among the students.

VII semester Lab Manual

BIG DATA ANALYTICS (BIS701)

Dr. Prasanna Lakshmi G S
Associate Professor
Dept. of ISE

Prof. Vidya H A
Assistant Professor
Dept. of ISE

&

PARALLEL COMPUTING (BCS702)

Prof. Shivaram A M
Associate Professor
Dept. of ISE

Prof. Radha R
Assistant Professor,
Dept. of ISE

Name: _____

USN : _____



SAI VIDYA INSTITUTE OF TECHNOLOGY

(Affiliated to VTU, Belagavi, Approved by AICTE, New Delhi and Govt. of Karnataka)

Accredited by NBA, New Delhi (CSE, ISE, ECE), NAAC – ‘A’ Grade

Rajanukunte, Bengaluru-560064

Tel: 080-2846 8196, Fax: 2846 8193 / 98, Web: www.saividya.ac.in



MOTTO

“Learn to lead”

VISION

Contribute dedicated, skilled, intelligent engineers and business administrators to architect strong India and the world.

MISSION

M1: To provide quality education and skilled training to produce dedicated engineers and managers.

M2: To promote research, innovation, and ethical practices by creating supportive environment.

M3: To undertake collaborative projects with academia and industry that transform young minds into Socially Responsible Citizen and Globally Competent Professionals.

M4: To enhance personality traits which lead to entrepreneurship qualities among the students.

BIG DATA ANALYTICS (BIS701)

(As per Visvesvaraya Technological University Syllabus)

Compiled by:

Dr. Prasanna Lakshmi G S
Associate Professor
Dept. of ISE

Prof. Vidya H A
Assistant Professor
Dept. of ISE

DEPARTMENT OF INFORMATION SCIENCE AND ENGINEERING

2025-26

Disclaimer

The information contained in this document is the proprietary and exclusive property of Sai Vidya Institute of Technology except as otherwise indicated. No part of this document, in whole or in part, may be reproduced, stored, transmitted, or used for course material development purposes without the prior written permission of Sai Vidya Institute of Technology. The information contained in this document is subject to change without notice. The information in this document is provided for informational purposes only.

Trademark Edition: 2025-26



Document Owner

The primary contacts for questions regarding this document are:

Author(s):

1. Dr. Prasanna Lakshmi G S
2. Prof. Vidya H A

Department:

Information Science and Engineering

Email Id's:

prasannalakshmi.gs@saividya.ac.in

vidya.ha@saividya.ac.in

DEPARTMENT OF INFORMATION SCIENCE & ENGINEERING

DEPARTMENT VISION

Architect dedicated and proficient engineers to address evolving global information technology needs.

DEPARTMENT MISSION

M1: To provide quality education and skilled training to produce dedicated engineers and ethical coders.

M2: To prepare young minds for the competitive world through industry collaborative projects and research practices to address societal needs.

M3: To enhance personality traits which lead to entrepreneurship qualities among the students.

DEPARTMENT PROGRAM EDUCATIONAL OBJECTIVE

PEO 1: *Serve as IT professional with proficiency in developing solutions to Complex Engineering problems.*

PEO 2: *Pursue higher education, research and preserve the essence of lifelong learning.*

PEO 3: *Exhibit high standards of social and professional ethics, entrepreneurship and leadership qualities.*

Program Outcomes

- 1. Engineering Knowledge:** Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems..
- 2. Problem Analysis:** Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)
- 3. Design/Development of Solutions:** Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
- 4. Conduct Investigations of Complex Problems:** Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).
- 5. Engineering Tool Usage:** Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modeling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
- 6. The Engineer and The World:** Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).
- 7. Ethics:** Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
- 8. Individual and Collaborative Team work:** Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
- 9. Communication:** Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.
- 10. Project Management and Finance:** Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
- 11: Life-Long Learning:** Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.

Program Specific Outcomes

- PSO 1:** Analyze, design and develop secure Information system by organizing and visualizing data efficiently.
- PSO2:** Analyze, design, develop, test and maintain software that satisfies the specified requirements.
- PSO3:** Apply the knowledge of network communication concepts, computation, optimization & innovative techniques to provide solutions to IT and societal problem.

Big Data Analytics (BIS701)

BIG DATA ANALYTICS		Semester	VII
Course Code	BIS701	CIE Marks	50
Teaching Hours/Week (L:T:P: S)	3:0:2:0	SEE Marks	50
Total Hours of Pedagogy	40 hours Theory + 8-10 Lab slots	Total Marks	100
Credits	04	Exam Hours	3
Examination nature (SEE)	Theory/practical		
Course objectives: 1. To implement MapReduce programs for processing big data. 2. To realize storage and processing of big data using MongoDB, Pig, Hive and Spark. 3. To analyze big data using machine learning techniques.			
Teaching-Learning Process (General Instructions) These are sample Strategies; that teachers can use to accelerate the attainment of the various course outcomes. 1. Lecturer method (L) needs not to be only a traditional lecture method, but alternative effective teaching methods could be adopted to attain the outcomes. 2. Use of Video/Animation to explain functioning of various concepts. 3. Encourage collaborative (Group Learning) Learning in the class. 4. Ask at least three HOT (Higher order Thinking) questions in the class, which promotes critical thinking. 5. Discuss how every concept can be applied to the real world - and when that's possible, it helps improve the students' understanding. 6. Use any of these methods: Chalk and board, Active Learning, Case Studies.			
MODULE-1			
Classification of data, Characteristics, Evolution and definition of Big data, What is Big data, Why Big data, Traditional Business Intelligence Vs Big Data, Typical data warehouse and Hadoop environment. Big Data Analytics: What is Big data Analytics, Classification of Analytics, Importance of Big Data Analytics, Technologies used in Big data Environments, Few Top Analytical Tools , NoSQL, Hadoop.			
TB1: Ch 1: 1.1, Ch2: 2.1-2.5,2.7,2.9-2.11, Ch3: 3.2,3.5,3.8,3.12, Ch4: 4.1,4.2			
MODULE-2			
Introduction to Hadoop: Introducing hadoop, Why hadoop, Why not RDBMS, RDBMS Vs Hadoop, History of Hadoop, Hadoop overview, Use case of Hadoop, HDFS (Hadoop Distributed File System), Processing data with Hadoop, Managing resources and applications with Hadoop YARN(Yet Another Resource Negotiator). Introduction to Map Reduce Programming: Introduction, Mapper, Reducer, Combiner, Partitioner, Searching, Sorting, Compression.			
TB1: Ch 5: 5.1-,5.8, 5.10-5.12, Ch 8: 8.1 - 8.8			
MODULE-3			

Introduction to MongoDB: What is MongoDB, Why MongoDB, Terms used in RDBMS and MongoDB, Data Types in MongoDB, MongoDB Query Language.

TB1: Ch 6: 6.1-6.5

MODULE-4

Introduction to Hive: What is Hive, Hive Architecture, Hive data types, Hive file formats, Hive Query Language (HQL), RC File implementation, User Defined Function (UDF).

Introduction to Pig: What is Pig, Anatomy of Pig, Pig on Hadoop, Pig Philosophy, Use case for Pig, Pig Latin Overview, Data types in Pig, Running Pig, Execution Modes of Pig, HDFS Commands, Relational Operators, Eval Function, Complex Data Types, Piggy Bank, User Defined Function, Pig Vs Hive.

TB1: Ch 9: 9.1-9.6,9.8, Ch 10: 10.1 - 10.15, 10.22

MODULE-5

Spark and Big Data Analytics: Spark, Introduction to Data Analysis with Spark.

Text, Web Content and Link Analytics: Introduction, Text Mining, Web Mining, Web Content and Web

PRACTICAL COMPONENT OF IPCC

Sl.NO	Experiments (Java/Python/R)
1	Install Hadoop and Implement the following file management tasks in Hadoop: Adding files and directories Retrieving files Deleting files and directories. Hint: A typical Hadoop workflow creates data files (such as log files) elsewhere and copies them into HDFS using one of the above command line utilities.
2	Develop a MapReduce program to implement Matrix Multiplication
3	Develop a Map Reduce program that mines weather data and displays appropriate messages indicating the weather conditions of the day.
4	Develop a MapReduce program to find the tags associated with each movie by analyzing movie lens data.
5	Implement Functions: Count – Sort – Limit – Skip – Aggregate using MongoDB
6	Write Pig Latin scripts to sort, group, join, project, and filter the data.
7	Use Hive to create, alter, and drop databases, tables, views, functions, and indexes.
8	Implement a word count program in Hadoop and Spark.
9	Use CDH (Cloudera Distribution for Hadoop) and HUE (Hadoop User Interface) to analyze data and generate reports for sample datasets

Course outcomes (Course Skill Set):

At the end of the course, the student will be able to:

- Identify and list various Big Data concepts, tools and applications.
- Develop programs using HADOOP framework.
- Apply NoSQL databases like MongoDB for storing, querying, and managing unstructured and semi-structured data.
- Use Hadoop Cluster to deploy Map Reduce jobs, PIG, HIVE and Spark programs.
- Analyze the given data set and identify deep insights from the data set.

Assessment Details (both CIE and SEE)

The weightage of Continuous Internal Evaluation (CIE) is 50% and for Semester End Exam (SEE) is 50%. The minimum passing mark for the CIE is 40% of the maximum marks (20 marks out of 50) and for the SEE minimum passing mark is 35% of the maximum marks (18 out of 50 marks). A student shall be deemed to have satisfied the academic requirements and earned the credits allotted to each subject/ course if the student secures a minimum of 40% (40 marks out of 100) in the sum total of the CIE (Continuous Internal Evaluation) and SEE (Semester End Examination) taken together.

CIE for the theory component of the IPCC (maximum marks 50)

- IPCC means practical portion integrated with the theory of the course.
- CIE marks for the theory component are **25 marks** and that for the practical component is **25 marks**.
- 25 marks for the theory component are split into **15 marks** for two Internal Assessment Tests (Two Tests, each of 15 Marks with 01-hour duration, are to be conducted) and **10 marks** for other assessment methods mentioned in 22OB4.2. The first test at the end of 40-50% coverage of the syllabus and the second test after covering 85-90% of the syllabus.

CONTENT BEYOND SYLLABUS

1. Develop a Map Reduce program for sample health data monitoring.
2. AIM: Develop a Map Reduce program for sample retail data

Program 1

Install Hadoop and implement the following file management tasks in Hadoop

Adding Files and Directories

Retrieving Files

Delete files and directories

Hint: A typical Hadoop workflow creates data files (such as log files) elsewhere and copies them into HDFS using one of the above command line utilities.

Source code:

Download and install Oracle virtual box & Cloudera and execute the following commands in terminal

Note: Make sure Java version 1.8 is installed in your system before the installation of hadoop.

1. Download Hadoop Binary Package:

- Visit the Hadoop releases page and download the latest binary package.
- Extract the downloaded package to a directory (e.g., C:\hadoop-3.1.2).

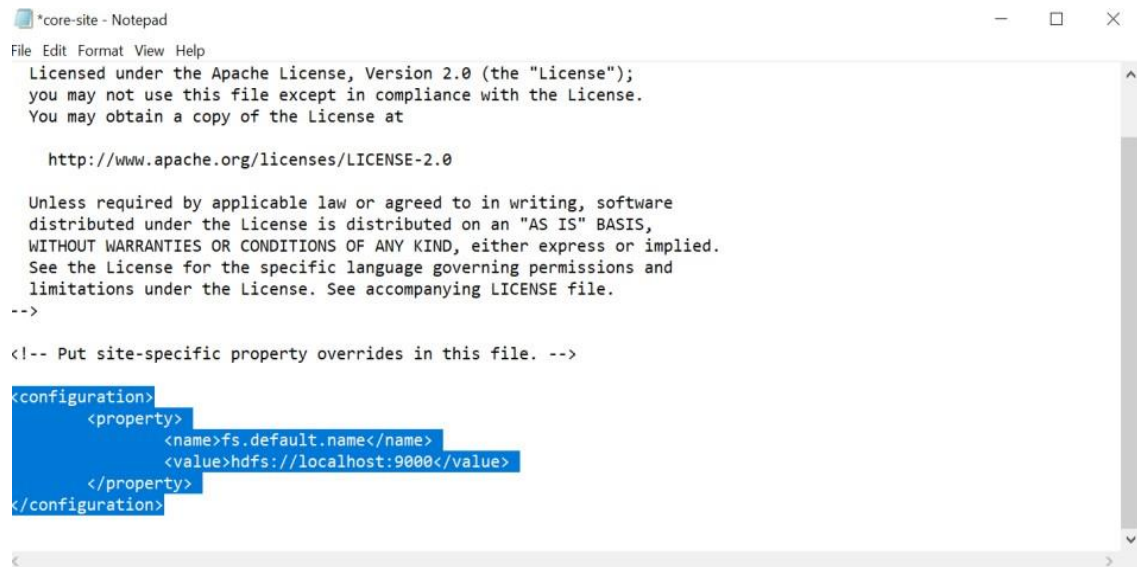
2. Set Environment Variables:

Use the below path to access the different sites of hadoop:

C:\hadoop-3.4.1\etc\hadoop

First open the core-site with notepad and type the following commands and save it.

```
<configuration>
  <property>
    <name>fs.default.name</name>
    <value>hdfs://localhost:9000</value>
  </property>
</configuration>
```



```

*core-site - Notepad
File Edit Format View Help
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at

    http://www.apache.org/licenses/LICENSE-2.0

Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License. See accompanying LICENSE file.
-->

<!-- Put site-specific property overrides in this file. -->

<configuration>
  <property>
    <name>fs.default.name</name>
    <value>hdfs://localhost:9000</value>
  </property>
</configuration>

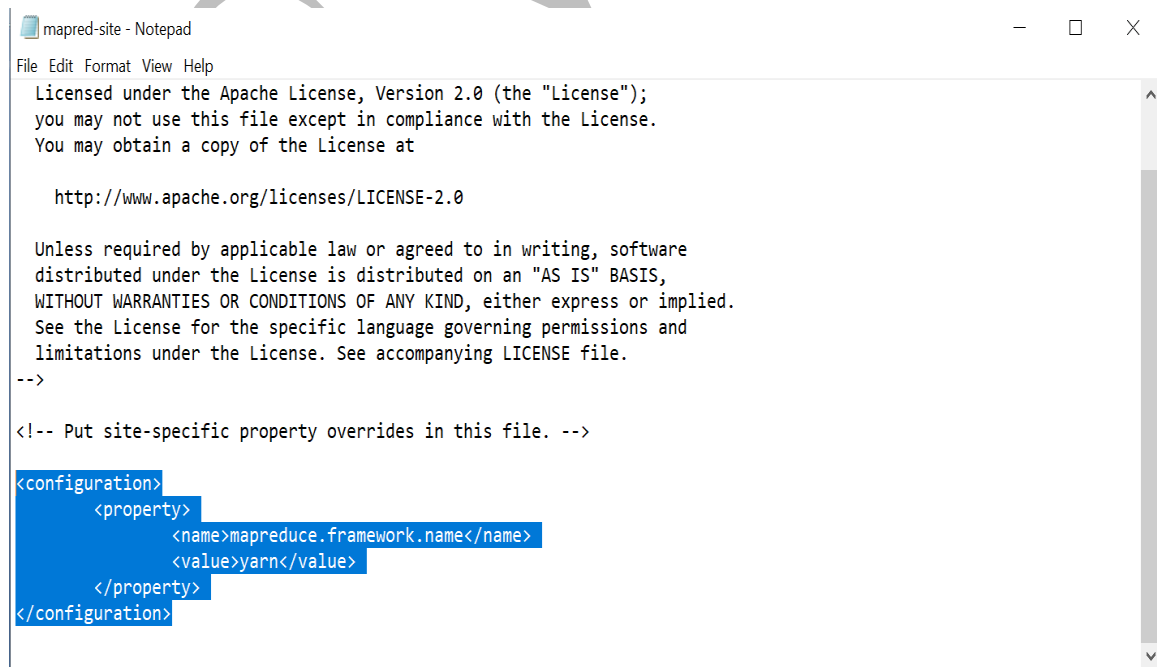
```

Next open the mapred-site with notepad and type the following commands and save it.

```

<configuration>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
</configuration>

```



```

mapred-site - Notepad
File Edit Format View Help
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at

    http://www.apache.org/licenses/LICENSE-2.0

Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License. See accompanying LICENSE file.
-->

<!-- Put site-specific property overrides in this file. -->

<configuration>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
</configuration>

```

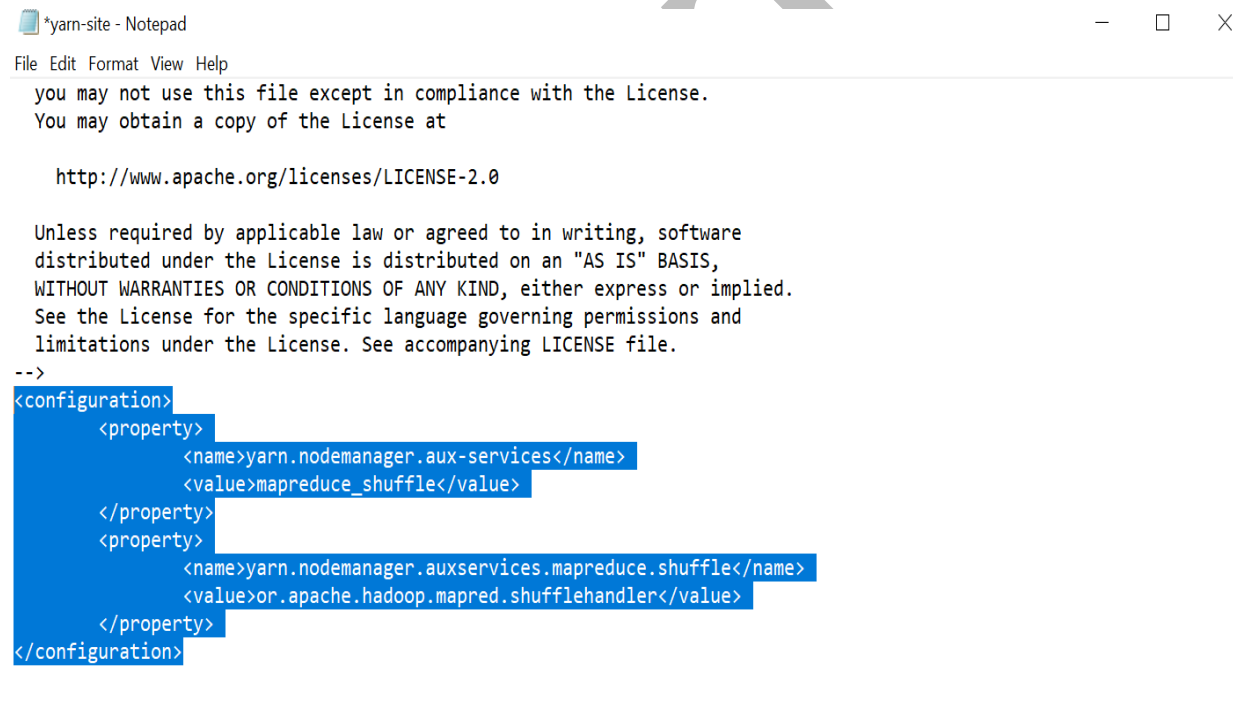
Next open the yarn-site with notepad and type the following commands and save it.

```
<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>

    <value>mapreduce_shuffle</value>

  </property>
  <property>
    <name>yarn. nodemanager.auxservices.mapreduce.shuffle</name>
    <value>or.apache.hadoop.mapred.shufflehandler</value>

  </property>
</configuration>
```



Next open the hdfs-site with notepad and type the following commands and save it. Before typing the commands create 2 new folders with the name **datanode** and **namenode** inside the new folder data created in the following path:

C:\hadoop-3.4.1\data

This PC > Windows (C:) > hadoop-3.4.1 > data			
☐	Name	Date modified	Type
	datanode	04-Mar-25 9:21 AM	File folder
	namenode	04-Mar-25 12:59 PM	File folder

<configuration>

<property>

<name>dfs.replication</name>

<value>1</value>

</property>

<property>

<name>dfs.namenode.name.dir</name>

<value>C:/hadoop-3.4.1/data/namenode</value>

</property>

<property>

<name>dfs.datanode.data.dir</name>

<value>C:/hadoop-3.4.1/data/datanode</value>

</property>

<property>

<name>dfs.datanode.du.reserved</name>

<value>1073741824</value>

</property>

</configuration>



```
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>

  <property>
    <name>dfs.namenode.name.dir</name>
    <value>C:/hadoop-3.4.1/data/namenode</value>
  </property>

  <property>
    <name>dfs.datanode.data.dir</name>
    <value>C:/hadoop-3.4.1/data/datanode</value>
  </property>

  <property>
    <name>dfs.datanode.du.reserved</name>
    <value>1073741824</value>
  </property>
</configuration>
```

- Open hadoop-env with note pad and make the following changes: set
JAVA_HOME=%JAVA_HOME%
set JAVA_HOME=C:\Java\jdk1.8.0_202
set JAVA_BIN=%JAVA_HOME%\bin
set HADOOP_HOME=%HADOOP_HOME%
set HADOOP_HOME=C:\hadoop-3.4.1
SET HADOOP_BIN=%HADOOP_HOME%\bin
set HADOOP_CONF_DIR=C:\hadoop-3.4.1\etc\hadoop



```

hadoop-env - Notepad
File Edit Format View Help

@rem Set Hadoop-specific environment variables here.

@rem The only required environment variable is JAVA_HOME. All others are
@rem optional. When running a distributed configuration it is best to
@rem set JAVA_HOME in this file, so that it is correctly defined on
@rem remote nodes.

@rem The java implementation to use. Required.
set JAVA_HOME=%JAVA_HOME%
set JAVA_HOME=C:\Java\jdk1.8.0_202
set JAVA_BIN=%JAVA_HOME%\bin
set HADOOP_HOME=%HADOOP_HOME%
set HADOOP_HOME=C:\hadoop-3.4.1
SET HADOOP_BIN=%HADOOP_HOME%\bin
set HADOOP_CONF_DIR=C:\hadoop-3.4.1\etc\hadoop

@rem The jsvc implementation to use. Jsvc is required to run secure datanodes.
@rem set JSVC_HOME=%JSVC_HOME%

@rem Extra Java CLASSPATH elements. Automatically insert capacity-scheduler.
if exist %HADOOP_HOME%\contrib\capacity-scheduler (

```

- Set the following environment variables:
 - HADOOP_HOME: Path to the Hadoop installation directory (e.g., C:\hadoop-3.1.2).
 - HADOOP_BIN: Path to the Hadoop binary directory (e.g., C:\hadoop-3.1.2\bin).
 - JAVA_HOME: Path to your JDK installation (ensure it's version 1.8)

3. Edit PATH Environment Variable:

- o Add %HADOOP_HOME%, %HADOOP_BIN%, and %JAVA_HOME%\bin to your system's PATH variable.

4. Configure Hadoop:

- o Edit the Hadoop configuration files (e.g., core-site.xml, hdfs-site.xml, etc.) to specify the paths and settings.

➤ Create a directory in HDFS at given path(s) Syntax:

C:\hadoop>hadoop fs -mkdir /Directory1

```
C:\Windows\System32\cmd.e X + v
Microsoft Windows [Version 10.0.22621.3296]
(c) Microsoft Corporation. All rights reserved.

C:\hadoop>hadoop fs -mkdir /Directory1

C:\hadoop>
```

- List contents of a directory

Syntax: C:\hadoop fs -ls /Directory1

```
C:\hadoop>hadoop fs -ls /Directory1

C:\hadoop>
```

- Upload the download a file in HDFS

Copy single src file, or multiple src files from local system to the Hadoop file system

Syntax: C:\hadoop>Hadoop fs -put D:\Nakka.txt/Directory1

```
C:\Windows\System32\cmd.e X + v
Microsoft Windows [Version 10.0.22621.3296]
(c) Microsoft Corporation. All rights reserved.

C:\hadoop>hadoop fs -put D:\Nakka.txt /Directory1

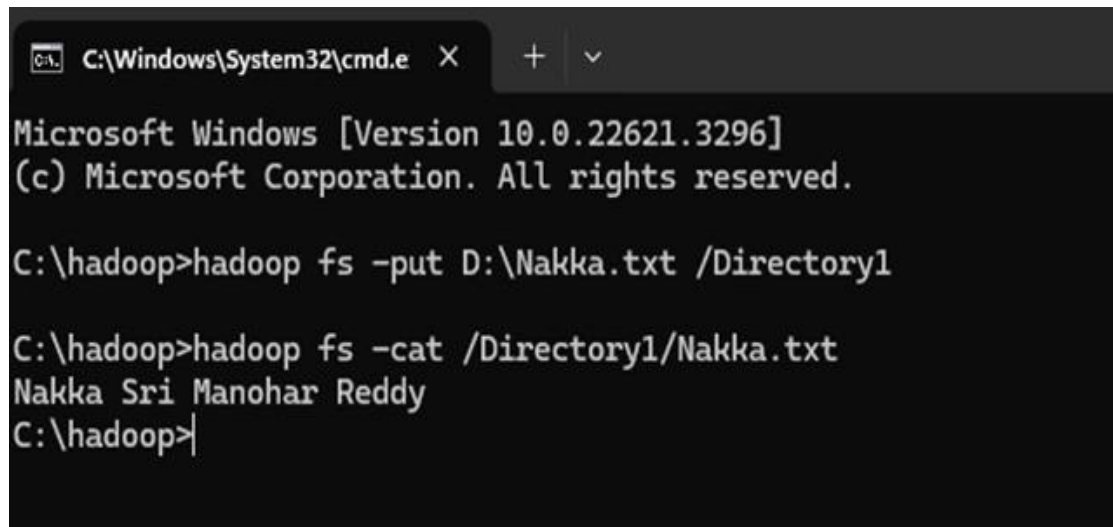
C:\hadoop>
```

- Copies/Downloads files to the local file system

Syntax: C:\hadoop>Hadoop fs -get /Directory1 D:\Nakka.txt/Directory1

- Set the contents of a file

Syntax: C:\hadoop>hadoop fs -cat /Directory1/Nakka.txt



```

C:\Windows\System32\cmd.e X + v
Microsoft Windows [Version 10.0.22621.3296]
(c) Microsoft Corporation. All rights reserved.

C:\hadoop>hadoop fs -put D:\Nakka.txt /Directory1

C:\hadoop>hadoop fs -cat /Directory1/Nakka.txt
Nakka Sri Manohar Reddy
C:\hadoop>

```

- Copy a file to from source to destination
- This command allows multiple sources as well in which case the destination must be a directory.

Syntax: C:\hadoop>hadoop fs -mkdir /Directory2

Syntax: C:\hadoop>hadoop fs -mkdir/Directory3

- Copy a file from/To Local file system to HDFS

Copy To Local

Similar to put command, except that the source is restricted to a local file reference.

Syntax: C:\hadoop>hadoop fs -cp /Directory1/Nakka.txt/Directory2

- **Remove a file or directory in HDFS**

Remove files specified as argument. Deletes directory only when it is empty.

Syntax: C:\hadoop>hadoop fs -rm /Directory2/Nakka.txt

```
C:\hadoop>hadoop fs -rm /Directory2/Nakka.txt
Deleted /Directory2/Nakka.txt

C:\hadoop>
```

- Display the aggregate length of a file

Syntax: C:\hadoop>hadoop fs -du /Directory1

- Removing a Directory

Syntax: C:\hadoop>hadoop fs -rm -r /Directory3

SVIT

Program 2

Develop a Map Reduce program for Matrix Multiplication.

Execute in Jupiter/Spyder

```
from functools import reduce

def mapper(matrix1, matrix2):
    key_value_pairs = []
    for i in range(len(matrix1)):
        for j in range(len(matrix2[0])):
            for k in range(len(matrix2)):
                key_value_pairs.append(((i, j), matrix1[i][k] * matrix2[k][j]))
    return key_value_pairs

def reducer(key, values):
    return sum(values)

def map_reduce(matrix1, matrix2):
    key_value_pairs = mapper(matrix1, matrix2)
    grouped_values = {}
    for key, value in key_value_pairs:
        if key not in grouped_values:
            grouped_values[key] = []
        grouped_values[key].append(value)
    result = {}
    for key in grouped_values:
        result[key] = reducer(key, grouped_values[key])
    return result

def result_to_matrix(result, rows, cols):
    matrix = [[0 for _ in range(cols)] for _ in range(rows)]
    for key, value in result.items():
        matrix[key[0]][key[1]] = value
    return matrix
```

```
matrix1 = [[1, 2, 3],  
           [4, 5, 6]]
```

```
matrix2 = [[7, 8],  
           [9, 10],  
           [11, 12]]
```

```
result = map_reduce(matrix1, matrix2)  
result_matrix = result_to_matrix(result, len(matrix1), len(matrix2[0]))  
for row in result_matrix:  
    print(row)
```

Sample Output:

```
[58, 64]
```

```
[139, 154]
```

Program 3

Develop a map reduce program that mines weather data and display appropriate message indicating the weather conditions of the day.

Execute in Jupiter/Spyder Environment

Step 1: Create a sample weather.txt file (you can edit this content)

```
with open('weather.txt', 'w') as f:
```

```
    f.write("""2024-04-01,Sunny
```

```
2024-04-01,Rainy
```

```
2024-04-01,Sunny
```

```
2024-04-02,Cloudy
```

```
2024-04-02,Cloudy
```

```
2024-04-02,Rainy
```

```
2024-04-03,Sunny
```

```
""")
```

```
print("weather.txt file created.")
```

Step 2: Define the mapper function

```
def mapper(line):
```

```
    # Split each line into date and weather condition
```

```
    date, condition = line.strip().split(',')
```

```
    return (date, condition)
```

Step 3: Define the reducer function

```
def reducer(date, conditions):
```

```
    # Count frequency of each weather condition
```

```
    condition_counts = {}
```

```
    for condition in conditions:
```

```
        if condition in condition_counts:
```

```
            condition_counts[condition] += 1
```

```
        else:
```

```
            condition_counts[condition] = 1
```

```
    # Get the condition with the maximum count
```

```
    most_frequent_condition = max(condition_counts, key=condition_counts.get)
```

```
    return (date, most_frequent_condition)
```

Step 4: Define the main function (for Jupyter, we'll call it manually)

```
def main(input_file):
```

```
    # Read input lines from the file
```

```
    with open(input_file, 'r') as f:
```

```
lines = f.readlines()

# Apply mapper to each line
mapped_data = [mapper(line) for line in lines]

# Group conditions by date
grouped_data = {}
for date, condition in mapped_data:
    if date in grouped_data:
        grouped_data[date].append(condition)
    else:
        grouped_data[date] = [condition]

# Apply reducer to grouped data
reduced_data = {date: reducer(date, conditions) for date, conditions in grouped_data.items()}

# Print final results
for date, (date_key, condition) in reduced_data.items():
    print(f"Weather on {date_key}: {condition}")

# Step 5: Run the main function with the sample file
main('weather.txt')
```

Sample Output:

```
Weather on 2024-04-01: Sunny
Weather on 2024-04-02: Cloudy
Weather on 2024-04-03: Sunny
```

Program 4

Develop a Map Reduce program to find the tags associated with each movie by analyzing movie lens data.

Execute in Jupiter/Spyder Environment

```
import csv
import os

# Step 1: Create sample CSV files
def create_sample_files():
    # Sample data for movies.csv
    movies_data = [
        ['movieId', 'title'],
        ['1', 'Toy Story'],
        ['2', 'Jumanji'],
        ['3', 'Grumpier Old Men']
    ]

    # Sample data for tags.csv
    tags_data = [
        ['userId', 'movieId', 'tag', 'timestamp'],
        ['10', '1', 'funny', '1234567890'],
        ['11', '1', 'animation', '1234567891'],
        ['12', '2', 'adventure', '1234567892'],
        ['13', '2', 'game', '1234567893'],
        ['14', '1', 'funny', '1234567894'],
        ['15', '3', 'romantic', '1234567895']
    ]

    # Write movies.csv
    with open('movies.csv', mode='w', newline='') as file:
        writer = csv.writer(file)
        writer.writerows(movies_data)

    # Write tags.csv
    with open('tags.csv', mode='w', newline='') as file:
        writer = csv.writer(file)
        writer.writerows(tags_data)
```

Step 2: Define MapReduce functions

```
def mapper(tags):
    key_value_pairs = []
    for row in tags:
        movie_id = row['movieId']
        tag = row['tag']
        key_value_pairs.append((movie_id, tag))
    return key_value_pairs

def reducer(key, values):
    return list(set(values)) # Remove duplicates

def map_reduce(tags):
    key_value_pairs = mapper(tags)

    grouped_values = {}
    for key, value in key_value_pairs:
        if key not in grouped_values:
            grouped_values[key] = []
        grouped_values[key].append(value)

    result = {}
    for key in grouped_values:
        result[key] = reducer(key, grouped_values[key])

    return result
```

Step 3: Load data from the CSV files

```
def load_tags(file_path):
    tags = []
    with open(file_path, mode='r') as file:
        reader = csv.DictReader(file)
        for row in reader:
            tags.append(row)
    return tags

def load_movies(file_path):
    movies = {}
    with open(file_path, mode='r') as file:
        reader = csv.DictReader(file)
        for row in reader:
```

```
movies[row['movieId']] = row['title']  
return movies
```

Step 4: Run everything

```
create_sample_files() # Create the files first
```

```
tags = load_tags('tags.csv')  
movies = load_movies('movies.csv')
```

```
result = map_reduce(tags)
```

Display the result

```
for movie_id, tags in result.items():  
    movie_title = movies.get(movie_id, "Unknown Title")  
    print(f'Movie: {movie_title}, Tags: {"", ".join(tags)}')
```

OUTPUT:

```
Movie: Toy Story, Tags: animation, funny  
Movie: Jumanji, Tags: adventure, game  
Movie: Grumpier Old Men, Tags: romantic
```

Program 5**Implement functions: Count-Sort-Limit-Skip- Aggregate using mango DB.**

step 1> mongosh

step 2> use experimentFiva

step 3>db.createCollection('myCollection')

step 4>

```
for (let i = 0; i < 10000; i++) {
  db.myCollection.insert({
    name: "Person" + i,
    age: Math.floor(Math.random() * 100),
    city: ["New York", "Los Angeles", "Chicago", "Houston", "Phoenix"][Math.floor(Math.random() * 5)]
  });
}
```

step 5> db.myCollection.find().count()

output> 10000

step 6> db.myCollection.find().skip(9999)

output> {

```
  _id: ObjectId('67d002f99941824b789fc01c'),
  name: 'Person9999',
  age: 45,
  city: 'Los Angeles'
```

}

step 7> db.myCollection.find().limit(2) output>

[

{

```
  _id: ObjectId('67cffc139941824b789f990a'),
  name: 'rahul',
  age: 32,
```

```

        city: 'Bangaluru'
      },
      {
        _id: ObjectId('67cffc8f9941824b789f990b'),
        name: 'XYZ',
        age: 38, city:
        'Bhalki'
      }
    ]
step 8> db.myCollection.find().count()

output> [
    {
      _id: ObjectId('67d002ef9941824b789f9b0b'),
      name: 'Person510',
      age: 0,
      city: 'Chicago'
    },
    {
      _id: ObjectId('67d002ef9941824b789f9b4b'),
      name: 'Person574',
      age: 0,
      city: 'Los Angeles'
    },
    .....
    ..
  ]

```

```
step 9> db.myCollection.find({age : {$gt : 99}})
```

```

output> [
  { _id: ObjectId('67cffc139941824b789f990a'),
    name: 'rahul',
    age: 32,

```

```
        city: 'Bengaluru'
    },
    {
        _id: ObjectId('67cffc8f9941824b789f990b'),
        name: 'XYZ',
        age: 38,
        city: 'Bhalki'
    },.....
]
```

SVIT

Program 6

Write a Pig Latin Scripts to sort, group, join, Project and filter the data.

1. Sample Data

Let's assume we have two datasets:

- employees.txt (Employee ID, Name, Age, Department ID, Salary)
- departments.txt (Department ID, Department Name)

employees.txt (stored in HDFS at /user/cloudera/employees.txt)

101, John,30,1,50000

102, Sam,28,2,60000

103, Anna,32,1,75000

104, David,29,3,62000

105, Lily,27,2,58000

departments.txt (stored in HDFS at /user/cloudera/departments.txt)

1, HR 2, Finance 3, IT

Pig Latin Script

Save the script as employee_analysis.pig and execute it in Cloudera.

-- Load the employees dataset

```
employees = LOAD 'hdfs://localhost:9000/user/cloudera/employees.txt' USING PigStorage(',')
AS (emp_id:int, name:chararray, age:int, dept_id:int, salary:int);
```

-- Load the departments dataset

```
departments = LOAD 'hdfs://localhost:9000/user/cloudera/departments.txt' USING PigStorage(',')
AS (dept_id:int, dept_name:chararray);
```

-- 1. FILTER: Select employees with age greater than 28 filtered_employees = FILTER employees BY age > 28;

-- 2. PROJECT: Select only emp_id, name, and salary columns projected_employees = FOREACH filtered_employees GENERATE emp_id, name, salary;

-- 3. SORT: Order employees by salary in descending order sorted_employees = ORDER projected_employees BY salary DESC;

-- 4. GROUP: Group employees by department ID grouped_by_department = GROUP employees BY dept_id;

-- 5. JOIN: Join employees with department names using dept_id joined_data = JOIN employees BY dept_id, departments BY dept_id;

-- STORE results in HDFS

STORE sorted_employees INTO

'hdfs://localhost:9000/user/cloudera/output/sorted_employees' USING PigStorage(',');

STORE grouped_by_department INTO

'hdfs://localhost:9000/user/cloudera/output/grouped_by_department' USING PigStorage(',');

STORE joined_data INTO 'hdfs://localhost:9000/user/cloudera/output/joined_data' USING PigStorage(',');

-- DISPLAY the results on the screen DUMP sorted_employees; DUMP grouped_by_department; DUMP joined_data;

Upload the data in HDFS

hdfs dfs -mkdir -p /user/cloudera

hdfs dfs -put employees.txt /user/cloudera/ hdfs dfs -put departments.txt /user/cloudera/

Run the Script

pig -x mapreduce employee_analysis.pig

output commands

hdfs dfs -cat /user/cloudera/output/sorted_employees/part-r-00000

hdfs dfs -cat /user/cloudera/output/grouped_by_department/part-r-00000 hdfs dfs

-cat /user/cloudera/output/joined_data/part-r-00000

OUTPUT:

```
103,Anna,75000
104,David,62000
102,Sam,60000
101,John,50000
```

Grouped Employees by Departments

```
(1, {(101, John, 30, 1, 50000), (103, Anna, 32, 1, 75000)})  
(2, {(102, Sam, 28, 2, 60000), (105, Lily, 27, 2, 58000)})  
(3, {(104, David, 29, 3, 62000)})
```

Joined Employees with Departments

```
(101, John, 30, 1, 50000, 1, HR)  
(102, Sam, 28, 2, 60000, 2, Finance)  
(103, Anna, 32, 1, 75000, 1, HR)  
(104, David, 29, 3, 62000, 3, IT)  
(105, Lily, 27, 2, 58000, 2, Finance)
```

Program 7

Use HIVE to create, alter, and drop databases, tables, views, functions and indexes

Create a Database:

```
CREATE DATABASE employee_db;
```

Output:

```
OK  
Time taken: 0.234 seconds
```

Use the database:

```
USE employee_db; Output:
```

```
OK  
Time taken: 0.123 seconds
```

Alter a Database:

```
ALTER DATABASE employee_db SET DBPROPERTIES ('Owner'='Admin');
```

```
OK  
Time taken: 0.134 seconds
```

Drop a Data Base

```
DROP DATABASE employee_db CASCADE;
```

```
OK  
Time taken: 0.321 seconds
```

Create a Table

```
CREATE TABLE employees ( emp_id INT,  
    name STRING, age INT, dept_id INT, salary FLOAT  
)
```

ROW FORMAT DELIMITED FIELDS TERMINATED BY ';' STORED AS TEXTFILE;

Output:

```
OK
Time taken: 0.567 seconds
```

LOAD DATA INPATH '/user/cloudera/employees.txt' INTO TABLE employees;

Output:

```
Loading data to table employee_db.employees
OK
Time taken: 1.543 seconds
```

Alter a Table

Add a New Column

ALTER TABLE employees ADD COLUMNS (email STRING); Output:

```
OK
Time taken: 0.234 seconds
```

Rename:

ALTER TABLE employees RENAME TO employees_new;

Drop Table:

DROP TABLE employees_new;

Create a View

CREATE VIEW high_salary_employees AS SELECT emp_id, name, salary
FROM employees WHERE salary > 50000;

Alter a View:

```
ALTER VIEW high_salary_employees AS SELECT emp_id, name, age, salary FROM  
employees  
WHERE salary > 60000;
```

Drop View:

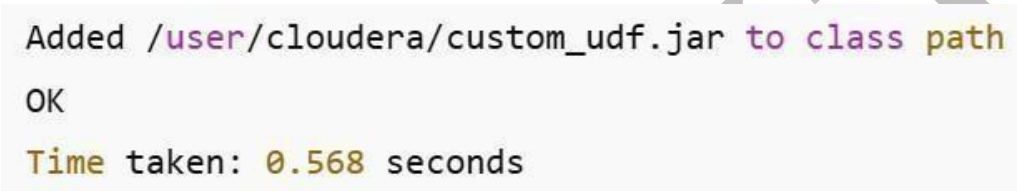
```
DROP VIEW high_salary_employees;
```

Create a Function

Add a JAR file containing a Java-based UDF:

```
ADD JAR /user/cloudera/custom_udf.jar;
```

```
CREATE FUNCTION to_upper AS 'com.example.hiveudf.ToUpperUDF';
```



```
Added /user/cloudera/custom_udf.jar to class path  
OK  
Time taken: 0.568 seconds
```

Use the Function

```
SELECT to_upper(name) FROM employees;
```

Output:

```
JOHN  
SAM  
ANNA  
DAVID  
LILY  
OK  
Time taken: 0.345 seconds
```

Drop Function

```
DROP FUNCTION to_upper;
```

Create an Index

```
CREATE INDEX emp_dept_idx ON TABLE employees (dept_id)
AS 'org.apache.hadoop.hive.ql.index.compact.CompactIndexHandler' WITH
DEFERRED REBUILD;
```

Output:

```
OK
Time taken: 0.765 seconds
```

Rebuild the Index:

```
ALTER INDEX emp_dept_idx ON employees REBUILD;
```

Drop Index

```
DROP INDEX emp_dept_idx ON employees;
```

Check all tables in the current database:

```
SHOW TABLES;
```

```
default
employee_db
OK
Time taken: 0.167 seconds
```

Check all tables in the current database: SHOW TABLES;

```
employees
employees_new
OK
Time taken: 0.145 seconds
```

DESCRIBE employees;

```
emp_id      int
name        string
age         int
dept_id     int
salary      float
email       string
OK
Time taken: 0.234 seconds
```

Display the table data

SELECT * FROM employees LIMIT 5;

```
101 John  30  1  50000.0  NULL
102 Sam   28  2  60000.0  NULL
103 Anna  32  1  75000.0  NULL
104 David 29  3  62000.0  NULL
105 Lily  27  2  58000.0  NULL
OK
Time taken: 0.459 seconds
```

Program 8**Implement word count program in Hadoop and spark Source Code.**

```
import org.apache.hadoop.conf. Configuration;

import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.IntWritable;

import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Job;

import org.apache.hadoop.mapreduce.Mapper;

import org.apache.hadoop.mapreduce.Reducer;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;

import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

import java.io.IOException;

import java.util.StringTokenizer;

public class WordCount {

    public static class TokenizerMapper extends Mapper<Object, Text, Text, IntWritable>
    {

        private final static IntWritable one = new IntWritable(1);
        private Text word = new Text();

        public void map(Object key, Text value, Context context)
            throws IOException, InterruptedException

        {

            StringTokenizer itr = new StringTokenizer(value.toString());

            while (itr.hasMoreTokens())

            {

                word.set(itr.nextToken());
```

```
        context.write(word, one);
    }
}

public static class IntSumReducer extends Reducer<Text, IntWritable, Text, IntWritable>
{
    public void reduce(Text key, Iterable<IntWritable> values, Context context)
        throws IOException, InterruptedException
    {
        int sum = 0;
        for (IntWritable val : values)
        {
            sum += val.get();
        }
        context.write(key, new IntWritable(sum));
    }
}

public static void main(String[] args) throws Exception
{
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "word count");
    job.setJarByClass(WordCount.class);
    job.setMapperClass(TokenizerMapper.class);
    job.setCombinerClass(IntSumReducer.class);
    job.setReducerClass(IntSumReducer.class);
```

```

job.setOutputKeyClass(Text.class);

job.setOutputValueClass(IntWritable.class);

FileInputFormat.addInputPath(job, new Path(args[0]));

FileOutputFormat.setOutputPath(job, new Path(args[1]));

System.exit(job.waitForCompletion(true) ? 0 : 1);

}

```

```

}

```

Compile and Package the Java Code

```

javac -classpath `hadoop classpath` -d . WordCount.java jar cf wc.jar
Wordcount*.class

```

Run the Hadoop Word Count Job Upload the input file to HDFS:

```

hdfs dfs -mkdir -p /user/cloudera/input
hdfs dfs -put sample.txt /user/cloudera/input/

```

Run the Map Reduce job

```

hadoop jar wc.jar WordCount /user/cloudera/input /user/cloudera/output

```

View the Output:

```

hdfs dfs -cat /user/cloudera/output/part-r-00000

```

```

Hadoop      2
Hello       3
MapReduce   1
Spark       2
World       2

```

Word Count in Apache Spark (PySpark)

1. Prepare the Input File

Upload the Sample.txt

```
hdfs dfs -mkdir -p /user/cloudera/input
```

```
hdfs dfs -put sample.txt /user/cloudera/input/ sample input File:
```

Hello Hadoop Hello Spark Hello World Spark is fast Hadoop is slow

2. Create a new script file using nano or vi:

```
nano wordcount.py
```

3. code

```
from pyspark.sql import SparkSession
```

```
# Initialize Spark Session
```

```
spark = SparkSession.builder.appName("WordCount").getOrCreate()
```

```
# Read text file from HDFS
```

```
text_file = spark.sparkContext.textFile("hdfs://localhost:9000/user/cloudera/input/sample.txt")
```

```
# Process data word_counts = (text_file  
    .flatMap(lambda line: line.split(" ")) # Split lines into words  
    .map(lambda word: (word, 1)) # Map each word to (word, 1)  
    .reduceByKey(lambda a, b: a + b)) # Reduce by key (word) and sum counts
```

```
# Save the output to HDFS
```

```
word_counts.saveAsTextFile("hdfs://localhost:9000/user/cloudera/output_spark")
```

```
# Print results
```

```
for word, count in word_counts.collect(): print(f"{word}: {count}") #
```

```
Stop Spark Session spark.stop()
```

3. Execute the Script

```
spark-submit wordcount.py
```

4. View the output:

```
hdfs dfs -ls /user/cloudera/output_spark
```

```
hdfs dfs -cat /user/cloudera/output_spark/part-00000
```

```
Hello 3  
Hadoop 2  
Spark 2  
World 1  
is 2  
fast 1  
slow 1
```

SVIT

Program 9

Use CDH (Cloudera Distribution for Hadoop) and HUE (Hadoop User Interface) to analyze the data and generate reports for sample data sets.

Steps and Source Code:

1. Start Cloudera services:

```
sudo service cloudera-scm-server start sudo service cloudera-scm-agent start
```

Check the status:

```
sudo service --status-all | grep cloudera
```

2. Access HUE Web Interface Open your browser and navigate to:

<http://localhost:8888>

Login using HUE credentials:

Username: cloudera Password: cloudera

3. Upload Sample Dataset to HDFS

Example Dataset: Employee Data (employees.csv) id,name,department,salary

1,John,IT,70000

2,Alice,HR,60000

3,Bob,IT,75000

4,Charlie,Finance,80000 5,David,HR,62000

6,Eva,IT,72000

7,Frank,Finance,81000 8,Grace,HR,65000

Upload employees.csv to HDFS using HUE

1. Navigate to HUE → File Browser → HDFS
2. Create a new directory /user/cloudera/data
3. Click **Upload** → Select employees.csv → Upload it Or through terminal

```
hdfs dfs -mkdir -p /user/cloudera/data
```

```
hdfs dfs -put employees.csv /user/cloudera/data/
```

Create a Hive Table in HUE

Open HUE → Click Query Editors → Select Hive

4. Create a Hive table for the dataset:

```
CREATE DATABASE IF NOT EXISTS company;
```

USE company;

```
CREATE TABLE employees ( id INT,
name STRING, department STRING, salary INT
)
```

ROW FORMAT DELIMITED FIELDS TERMINATED BY ',' STORED AS TEXTFILE;

Load data into the Hive table:

```
LOAD DATA INPATH '/user/cloudera/data/employees.csv' INTO TABLE employees;
```

Verify the data:

```
SELECT * FROM employees;
```

id	name	department	salary
1	John	IT	70000
2	Alice	HR	60000
3	Bob	IT	75000
4	Charlie	Finance	80000
5	David	HR	62000
6	Eva	IT	72000
7	Frank	Finance	81000
8	Grace	HR	65000

Data Analysis Using Hive Queries in HUE Find the Highest Salary in Each Department

```
SELECT department, MAX(salary) AS highest_salary FROM employees
GROUP BY department;
```

Output:

Department	Highest Salary
IT	75000
HR	65000
Finance	81000

Get Employees with Salary Greater Than 65000;**Output:**

Name	Department	Salary
John	IT	70000
Bob	IT	75000
Eva	IT	72000
Charlie	Finance	80000
Frank	Finance	81000

Generate Reports in HUE**Step 1: Export Query Results**

1. Run any of the above SQL queries in HUE Query Editor
2. Click Export → Choose format (CSV, Excel, JSON)
3. Download the report

Step 2: Create HUE Dashboard for Visualization

1. Open HUE → Click Dashboard
2. Click Create New Dashboard
3. Click Add Widget → Select Chart Type (Bar Chart, Pie Chart, etc.) Enter Query →
Example for Employee Count:

SELECT department, COUNT(*) AS employee_count FROM employees GROUP BY department;

Click "Run Query" → The visualization will be generated

Department	Employee Count
IT	3
HR	3
Finance	2

Viva Questions:

1. What is Hadoop and why is it used?
2. What are the main components of Hadoop?
3. What is HDFS? Explain its architecture.
4. What is the function of the NameNode in HDFS?
5. What is the function of the DataNode in HDFS?
6. What is the difference between NameNode and Secondary NameNode?
7. What is a block in HDFS, and how large is the default block size?
8. What is the function of YARN in Hadoop?
9. Explain the architecture of YARN.
10. What is MapReduce and how does it work?
11. What is a JobTracker in Hadoop 1.x?
12. What is a TaskTracker in Hadoop 1.x?
13. What is the difference between Hadoop 1.x and Hadoop 2.x?
14. What are the advantages of Hadoop over traditional RDBMS?
15. What is the role of ResourceManager and NodeManager in YARN?
16. What is data replication in HDFS? How is data redundancy handled?
17. What is the function of the JobHistory Server in Hadoop?
18. What are the main differences between HDFS and traditional file systems?
19. What is a combiner in MapReduce?
20. What is the difference between a combiner and a reducer?
21. What is MongoDB?
22. What are the advantages of using MongoDB over relational databases?
23. What are the data types supported by MongoDB?
24. What is a collection in MongoDB?
25. What is a document in MongoDB?
26. How does MongoDB handle data replication?
27. What is Sharding in MongoDB?
28. What is the use of an index in MongoDB?
29. What is the Aggregation framework in MongoDB?
30. How does MongoDB ensure fault tolerance?
31. What is the difference between MongoDB and RDBMS?

31. What are the different types of indexes in MongoDB?
32. What is the purpose of the find() method in MongoDB?
33. What is Hive and how is it different from traditional RDBMS?
34. What is the architecture of Hive?
35. What is HiveQL?
36. What is the function of the Hive Metastore?
37. What are the different types of tables in Hive?
38. What are the advantages of using Hive over SQL databases?
39. What are partitions in Hive?
40. What are buckets in Hive?
41. What is the difference between internal and external tables in Hive?
42. What file formats does Hive support?
43. What is Apache Spark?
44. What are the core components of Spark?
45. What is an RDD (Resilient Distributed Dataset) in Spark?
46. What are the advantages of using Spark over Hadoop MapReduce?
47. What is the role of Driver and Executor in Spark?
48. What are the types of transformations in Spark?
49. What are the types of actions in Spark?
50. What is Spark Streaming?
51. What is the purpose of Spark SQL?
52. What is the difference between map() and flatMap() in Spark?

CONTENT BEYOND SYLLABUS**1. Develop a Map Reduce program for sample health data monitoring**

```
health_data = [  
    (101, 25, 'M', 22.1),  
    (102, 30, 'F', 24.5),  
    (103, 22, 'M', 21.8),  
    (104, 25, 'F', 23.0),  
    (105, 30, 'M', 27.5),  
    (106, 22, 'F', 21.9),  
    (107, 30, 'M', 28.0),  
    (108, 25, 'F', 22.8)  
]  
  
def mapper(data):  
    output = []  
    for entry in data:  
        age = entry[1]  
        bmi = entry[3]  
        output.append((age, bmi))  
    return output  
  
def shuffle_and_sort(mapped_data):  
    from collections import defaultdict  
    grouped_data = defaultdict(list)  
    for age, bmi in mapped_data:  
        grouped_data[age].append(bmi)  
    return grouped_data  
  
def reducer(grouped_data):
```

```
averages = {}  
for age, bmi_list in grouped_data.items():  
    avg_bmi = sum(bmi_list) / len(bmi_list)  
    averages[age] = round(avg_bmi, 2)  
return averages
```

```
mapped_data = mapper(health_data)  
grouped_data = shuffle_and_sort(mapped_data)  
result = reducer(grouped_data)  
  
print("Average BMI per Age Group:")
```

```
print(result)
```

Output:

```
{  
    25: 22.5,  
    30: 26.0,  
    22: 21.85  
}
```

Program 2

Develop a Map Reduce program for sample retail data analysis.

```
retail_data = [  
    (101, 'Electronics', 200, 2),  
    (102, 'Clothing', 50, 3),  
    (103, 'Electronics', 150, 4),  
    (104, 'Clothing', 30, 5),  
    (105, 'Groceries', 20, 10),  
    (106, 'Groceries', 15, 7),  
    (107, 'Clothing', 60, 2)  
]  
  
def mapper(data):  
    output = []  
    for entry in data:  
        product_category = entry[1]  
        sales_amount = entry[2]  
        quantity_sold = entry[3]  
        total_sales = sales_amount * quantity_sold  
        output.append((product_category, total_sales))  
    return output  
  
def shuffle_and_sort(mapped_data):  
    from collections import defaultdict  
    grouped_data = defaultdict(list)  
    for product_category, total_sales in mapped_data:  
        grouped_data[product_category].append(total_sales)  
    return grouped_data  
  
def reducer(grouped_data):  
    total_sales_per_category = {}  
    for category, sales_list in grouped_data.items():  
        total_sales_per_category[category] = sum(sales_list)
```

```
return total_sales_per_category
```

```
mapped data = mapper (retail data)
```

```
grouped data = shuffle_and_sort (mapped data)
```

```
result = reducer (grouped data)
```

```
print ("Total Sales per Product Category:")
```

```
print(result)
```

Output:

```
{  
  'Electronics': 1000,  
  'Clothing': 510,  
  'Groceries': 260  
}
```